

Folder Mapping and Purpose Table

Complete Folder Structure and Purpose

Folder	Purpose	Key Technologies	Dependencies	File Types
UI/	Frontend user interfaces and components	React, Next.js, TypeScript, Tailwind CSS	Node.js, npm/yarn	.tsx, .jsx, .css, .js, .ts
RAG/	Retrieval Augmented Generation system	ChromaDB, Sentence Transformers, LangChain	Python, AI models	.py, .db files
AGENTS/	AI agents and orchestration	CrewAI, LangChain, OpenAI	RAG/, SERVICES/	.py
API/	Backend REST API and endpoints	FastAPI, Uvicorn, Pydantic	MODELS/, SERVICES/	.py
MODELS/	Database models and schemas	SQLAlchemy, Alembic	Database engine	.py, .sql
SERVICES/	Business logic and core services	Python, async/await	MODELS/, UTILS/	.py
UTILS/	Utility functions and helpers	Python standard library	None (base utilities)	.py
CONFIG/	Configuration files and settings	YAML, JSON, TOML	All components	.yaml, .json, .toml, .ini
TESTS/	Test files and validation scripts	pytest, FastAPI TestClient	All components	.py (test_*.py)
DOCS/	Documentation and guides	Markdown, reStructuredText	None	.md, .rst, .txt
PREREQUISITES/	Dependencies and requirements	pip, npm, Docker	None	.txt, .json, Dockerfile
MISC/	Miscellaneous files	Various	Various	Various

Dependency Relationships

Primary Dependencies

```

API/ → MODELS/ → Database
  |
  ├── SERVICES/ → UTILS/
  └── AGENTS/ → RAG/ → ChromaDB

UI/ → API/ (HTTP requests)

TESTS/ → All components (for testing)

```

Configuration Dependencies

```

All Components → CONFIG/ → PREREQUISITES/
                  |
                  └── Environment Variables

```

Detailed Folder Analysis

UI/ - Frontend Components

Purpose: User interface and user experience

- **Primary Files:** React components, pages, styles
- **Key Dependencies:** API/ for data, CONFIG/ for settings
- **Typical Structure:**

```

UI/
├── components/    # Reusable UI components
├── pages/         # Application pages
├── styles/        # CSS and styling
├── hooks/         # Custom React hooks
└── utils/         # Frontend utilities

```

RAG/ - Knowledge Management

Purpose: Document storage, embedding, and retrieval

- **Primary Files:** Vector database, embedding models, search logic
- **Key Dependencies:** AI models, ChromaDB, document processors
- **Integration Points:** AGENTS/ for knowledge queries, API/ for document management

AGENTS/ - AI Orchestration

Purpose: Automated task execution and AI assistance

- **Primary Files:** Agent definitions, crew management, task orchestration
- **Key Dependencies:** RAG/ for knowledge, SERVICES/ for business logic
- **Agent Types:** Project Manager, Developer, QA Engineer, Technical Writer

API/ - Backend Services

Purpose: RESTful API and business logic exposure

- **Primary Files:** FastAPI routes, middleware, authentication

- **Key Dependencies:** MODELS/ for data, SERVICES/ for logic, AGENTS/ for AI features
- **Endpoints:** CRUD operations, file uploads, agent interactions

MODELS/ - Data Layer

Purpose: Database schema and data models

- **Primary Files:** SQLAlchemy models, database configuration
- **Key Dependencies:** Database engine (SQLite/PostgreSQL)
- **Models:** Project, Task, Document, Agent, User

SERVICES/ - Business Logic

Purpose: Core application logic and workflows

- **Primary Files:** Service classes, business rules, workflows
- **Key Dependencies:** MODELS/ for data access, UTILS/ for helpers
- **Services:** Project management, task orchestration, document processing

UTILS/ - Helper Functions

Purpose: Common utilities and helper functions

- **Primary Files:** File operations, validation, configuration helpers
- **Key Dependencies:** None (base utilities)
- **Categories:** File handling, validation, logging, configuration

CONFIG/ - Configuration Management

Purpose: Application configuration and settings

- **Primary Files:** YAML/JSON config files, environment templates
- **Key Dependencies:** PREREQUISITES/ for base requirements
- **Types:** Database config, API settings, AI model config

TESTS/ - Quality Assurance

Purpose: Testing and validation

- **Primary Files:** Unit tests, integration tests, validation scripts
- **Key Dependencies:** All components (for testing)
- **Test Types:** API tests, RAG tests, agent tests, validation scripts

DOCS/ - Documentation

Purpose: Project documentation and guides

- **Primary Files:** Setup guides, architecture docs, API documentation
- **Key Dependencies:** None (documentation only)
- **Document Types:** Setup guides, architecture, API docs, troubleshooting

PREREQUISITES/ - Dependencies

Purpose: Project dependencies and requirements

- **Primary Files:** requirements.txt, package.json, Docker files
- **Key Dependencies:** None (base requirements)
- **Files:** Python requirements, Node.js packages, Docker configuration

Data Flow Between Folders

User Request Flow

1. **UI/** → **API/** (HTTP request)

2. **API/** → **SERVICES/** (business logic)
3. **SERVICES/** → **MODELS/** (data access)
4. **MODELS/** → Database (data persistence)

AI-Powered Workflow

1. **API/** → **AGENTS/** (task assignment)
2. **AGENTS/** → **RAG/** (knowledge retrieval)
3. **RAG/** → ChromaDB (vector search)
4. **AGENTS/** → **SERVICES/** (task execution)

Configuration Flow

1. **PREREQUISITES/** → Environment setup
2. **CONFIG/** → Application configuration
3. All components → **CONFIG/** (settings access)

Setup Dependencies

Installation Order

1. **PREREQUISITES/** - Install dependencies
2. **CONFIG/** - Configure environment
3. **MODELS/** - Initialize database
4. **API/** - Start backend services
5. **UI/** - Start frontend (optional)
6. **TESTS/** - Validate system

Runtime Dependencies

- **Database:** Required by **MODELS/**, **API/**, **SERVICES/**
- **AI Services:** Required by **AGENTS/**, **RAG/**
- **Web Server:** Required by **API/**, **UI/**
- **File System:** Required by all components for logging and storage

Scalability Considerations

Horizontal Scaling

- **API/:** Multiple instances behind load balancer
- **AGENTS/:** Distributed agent execution
- **RAG/:** Distributed vector database
- **UI/:** CDN distribution

Vertical Scaling

- **MODELS/:** Database optimization
- **SERVICES/:** Memory and CPU optimization
- **RAG/:** Vector operation optimization
- **UTILS/:** Efficient algorithms

This mapping provides a comprehensive understanding of how each folder contributes to the overall system architecture and functionality.